

Week 16 – Full Problem Solving Practice

■ Learning Objectives

- Apply a systematic method to solve a full CCC Junior set.
- Practice reading, planning, coding, and testing under time.
- Reflect on strategies for J1–J3 difficulty.

■ General Strategy

- Skim all problems; start with easiest for quick points.
- Turn words → steps → code; identify inputs/outputs.
- Check edge cases and sample tests before finalizing.

Example Walkthrough – Sample J2/J3-Style

Problem: Read N strings. Print how many are palindromes ignoring spaces/case.

```
def is_pal(s):  
    t = ''.join(ch.lower() for ch in s if ch.isalnum())  
    return t == t[::-1]
```

```
data = ["racecar", "Hello", "Never odd or even"]  
print(sum(1 for s in data if is_pal(s))) # 2
```

■ Practice Set

Practice 1 – Frequency Tally

```
words = "to be or not to be".split()
freq = {}
for w in words: freq[w] = freq.get(w,0)+1
print(freq)
```

Practice 2 – Grid Simulation (2D)

```
grid = [[0,1,0],[1,1,0],[0,0,1]]
rows, cols = len(grid), len(grid[0])
dirs = [(-1,0),(1,0),(0,-1),(0,1)]
# count neighbors equal to 1
counts = [[0]*cols for _ in range(rows)]
for r in range(rows):
    for c in range(cols):
        cnt = 0
        for dr,dc in dirs:
            nr, nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==1:
                cnt += 1
        counts[r][c] = cnt
counts
```

■ Homework Challenge – Mini Set

- J1: Read 3 ints, output their sum and average (1 decimal).
- J2: Given string, output first index of each vowel or -1 if missing.
- J3: Given grid with obstacles, count reachable cells from (0,0) using BFS.

J1

```
a,b,c = 5,7,9
```

```
print(a+b+c, (a+b+c)/3)
```

J2

```
s = "abracadabra"
```

```
vowels = "aeiou"
```

```
print([s.find(v) for v in vowels])
```

J3 (BFS skeleton)

```
from collections import deque
```

```
grid = [
```

```
    ['.', '.', '#', '.'],
```

```
    ['#', '.', '.', '.'],
```

```
    ['.', '#', '.', '.']
```

```
]
```

```
rows, cols = len(grid), len(grid[0])
```

```
q = deque([(0,0)]); seen = {(0,0)}
```

```
dirs = [(-1,0), (1,0), (0,-1), (0,1)]
```

```
while q:
```

```
    r,c = q.popleft()
```

```
    for dr,dc in dirs:
```

```
        nr, nc = r+dr, c+dc
```

```
        if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]=='.' and (nr,nc) not in seen:
```

```
            seen.add((nr,nc)); q.append((nr,nc))
```

```
print(len(seen))
```